

FlavorMap Final Report

Course: CSE 220 - Web Programming

Group Members: Beyza Yüz (231401701), Nehir Nur Hutoğlu (231401048), Armita Tavakolshoar (241401906), Nehir Çinik (241401025)

GitHub Repository: <https://github.com/beyz100/FlavorMap>

1. Project Description

FlavorMap is a Django web application in which people discover restaurants, read and write reviews, and keep a personal list of their favorite places. The idea was based on a simplified version of platforms like Yelp or Google Reviews, but built from scratch using only the framework features taught during the course. The first weeks were spent on the basic skeleton (URL routing, templates, static views), then on real data with the database, and finally on forms, authentication, and the bonus features.

User Stories

The features were planned around the following short user stories:

- **As a visitor who has not signed up:** I want to browse restaurants, see their menus, and read reviews, so that I can decide where to eat without creating an account. I want to filter restaurants by category, location, and price range, so that I can quickly find places that match what I am looking for.
- **As a registered user:** I want to write a review with a star rating and a comment, so that I can share my opinion with others. I want to save restaurants to my favorites list, so that I can find them again later from my profile page. I want to reply to other users' reviews, so that conversations about a restaurant can happen on the page itself.
- **As a restaurant owner:** I want to add and manage menu items, opening hours, and photos for my restaurant, so that the page stays up to date.
- **As any user:** I want to switch the interface between English and Turkish, so that the website is accessible to both groups of users.

2. System Architecture

The project follows the Django Model-View-Template (MVT) pattern, split into two apps: a main restaurants app for core features and a users app for profile management and custom registration.

Models (M): Defined in app-specific models.py files to map database tables and relationships. Key features include custom `__str__` methods and dynamic calculations, such as the `average_rating()` method using Django's Avg aggregation.

Views (V): Implemented as function-based views that query the database and render templates. Access control is enforced using decorators like `@login_required` and `@require_POST`. Multi-step operations (e.g.,

submitting reviews and replies) are secured inside `transaction.atomic()` blocks with explicit `IntegrityError` handling to prevent crashes.

Templates (T): Built on a single `base.html` template that defines the navigation bar, flash messages, and footer. All other pages build upon it using `{% extends %}` and `{% block content %}` alongside common Django template tags.

URL Routing: Split modularly across the project-level and app-level `urls.py` files. The `restaurants` app handles approximately 25 endpoints for core features and CRUD actions, while the `users` app and Django's built-in `auth.urls` handle registration and authentication.

Static and Media Files: Interface styling is driven by a custom ~900-line CSS file. User-uploaded images are saved to a `media/` folder and served dynamically using `MEDIA_URL` and `MEDIA_ROOT` settings.

3. Database Design

The database uses SQLite, with the schema primarily centered around the `Restaurant` model.

Restaurant: Contains fields for `name`, `description`, `address`, `phone`, `price_range`, `photo`, and a Google Maps embed URL. It connects via `ForeignKey` to the built-in `User` (owner field for access control), `Category`, and `Location` (for filtering).

Category & Location: Simple lookup tables with a single name field each.

MenuItem: Linked via `ForeignKey` to `Restaurant`, containing `name`, `description`, `price`, and a free-text category.

Review: Linked via `ForeignKey` to `Restaurant` and `User`. It features a self-referencing parent `ForeignKey` so both top-level reviews and replies are modeled in a single table. A `UniqueConstraint` with the condition `parent__isnull=True` ensures a user can post only one main review per restaurant but unlimited replies.

ReviewLike: Links to `Review` and `User` with an `is_like` boolean. A `unique_together` constraint ensures a user can only have one reaction per review.

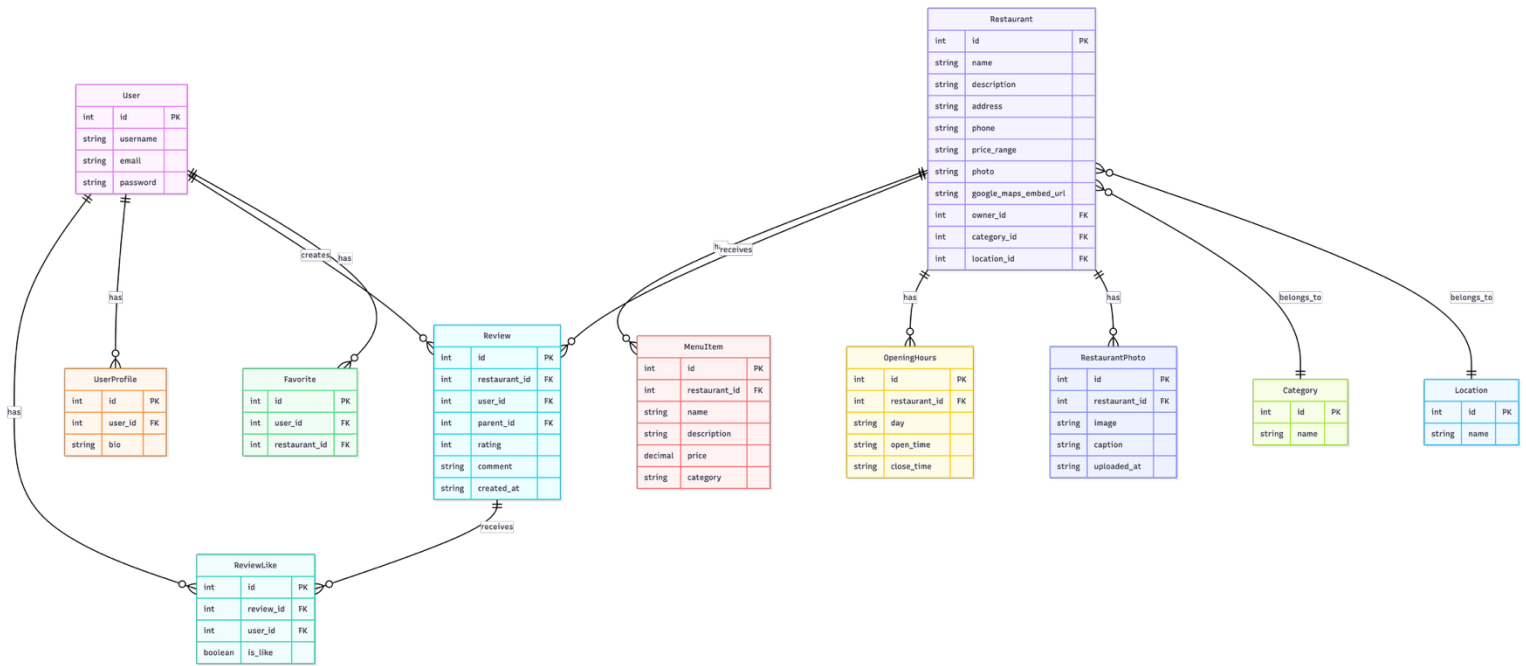
Favorite: Links `User` and `Restaurant`, utilizing a `UniqueConstraint` to prevent duplicate favorites.

OpeningHours: Linked to `Restaurant` with fields for `day`, `open_time`, and `close_time`. A `unique_together` constraint on `(restaurant, day)` prevents duplicate entries for the same day.

RestaurantPhoto: Linked to `Restaurant` using an `ImageField` and `caption` for the photo gallery.

UserProfile: Uses a `OneToOneField` to the `User` model to store a text bio.

ER Diagram:



4. Features

Mandatory (16/16):

Feature	Status	Notes
Restaurant CRUD	Done	create_restaurant, edit_restaurant, delete_restaurant views, with a separate confirmation page for deletion.
Category System	Done	Category model + dropdown filter on the list page.
Location Filter	Done	Location model + dropdown filter.
Price Range Filter	Done	Three choices (£, ££, £££) + dropdown filter.
Reviews & Ratings	Done	1–5 stars + comment + created_at. An interactive star widget was added.
Average Rating	Done	average_rating() method on the Restaurant model. List can also be sorted by rating using ?sort=rating.
Search	Done	Searches name, description, and location__name using Q objects with icontains.
User Authentication	Done	Custom registration form with email field, plus login and logout through Django's built-in auth.
Menu Management	Done	Add, edit, and delete menu items. Owner-only.
Favorites List	Done	Toggle favorite endpoint, list shown on profile page.
Restaurant Photo	Done	ImageField on the Restaurant model. MEDIA_URL/MEDIA_ROOT configured.
Opening Hours	Done	Seven-day choices, add/edit/delete views. The detail page lists them in correct weekday order using a Case/When expression.
Popular Ranking	Done	Home page shows the three top-rated restaurants and the three newest ones.

Feature	Status	Notes
User Profile	Done	Page shows the user's reviews, favorites, and a short bio.
Review Replies	Done	Implemented through the parent self-referencing ForeignKey on the Review model.
Atomic Transactions	Done	Used in add_review and add_reply. IntegrityError is caught and a clean message is shown.

Bonus (7/7):

Feature	Status	Notes
CSS & Responsive	Done	Custom stylesheet with CSS variables and a 768px responsive breakpoint.
JavaScript Element	Done	Interactive star rating widget. A photo lightbox was also added.
Restaurant Owner Role	Done	owner field plus a _user_owns_restaurant helper that gates all management views.
Map Integration	Done	A Google Maps iframe is embedded on the detail page using a per-restaurant URL field.
Photo Gallery	Done	RestaurantPhoto model with multiple photos per restaurant, plus a vanilla JS lightbox that opens on click and closes on Esc or click outside.
Advanced Filtering	Done	The list page combines category, location, price, rating sort all at once in a single GET form.
Review Likes	Done	ReviewLike model with like/dislike toggle. Reviews are sorted by like count on the detail page.

5. Challenges and Solutions

Merge conflicts on shared templates: Multiple group members editing the restaurant detail template at different times caused merge conflicts. We resolved this through stricter branching practices and, in one instance, reverting an accidental push to main to cleanly reapply the changes from the develop branch.

One review per user, but unlimited replies: Initially, a unique_together constraint on (restaurant, user) blocked users from posting replies, since replies are also stored as Review rows. The fix was implementing a UniqueConstraint with condition=Q(parent__isnull=True), which successfully restricts only top-level reviews while allowing unlimited replies.

Bilingual interface without Django i18n: To avoid the overhead of Django's gettext system (compiling .po/.mo files and configuring LocaleMiddleware), we built a lightweight JavaScript solution. Translatable elements use data-en and data-tr attributes swapped via a setLang() function, with choices saved in localStorage for cross-page consistency. Dates are localized via Intl.DateTimeFormat, with the acceptable trade-off that user-generated text remains untranslated.

6. Individual Contributions

Beyza Yüz (beyz100): Initializing the Django project, app structure, URL routing. Restaurant model and admin. Initial templates and most of the menu management views. Photo gallery upload feature. Bio field and the user profile page redesign. Lightbox for the gallery. Delete-confirmation page for restaurants. Final UI polish.

Nehir Nur Hutoğlu (nehirhutoglu): User authentication (login, logout, registration page, @login_required on the right views). Bilingual TR/EN translation system using data-en / data-tr attributes and the setLang() function. Several UI improvements. Sample gallery images. (**Note:** For 2 weeks, Nehir's commits were not tracked as her email on GitHub was neither valid nor verified.)

Armita Tavakolshoar (armita-41): User authentication. Restaurant ownership and the owner-only edit/delete authorization logic. The first styles.css and a consistent UI design across all restaurant templates. Detail page redesign. Dynamic views that read from the database. Restaurant creation URL pattern. README setup instructions.

Nehir Çinik (nehircinik): Review system: one review per user per restaurant, edit and delete own reviews, replies through the self-referencing parent ForeignKey. Atomic transactions for review and reply operations, with IntegrityError handling. Google Maps iframe integration. Favorites feature.

GitHub Actions (CI/CD) Pipeline: To ensure code reliability and automate our testing process, a GitHub Actions (CI/CD) workflow was integrated into the project, running our Django unit tests automatically on every commit.

GitHub Contribution Graphs from “Insights”:

